

PYIMAGESEARCH

DEEP LEARNING ([HTTPS://PYIMAGESEARCH.COM/CATEGORY/DEEP-LEARNING/](https://pyimagesearch.com/category/deep-learning/))

DEEPSEEK ([HTTPS://PYIMAGESEARCH.COM/CATEGORY/DEEPSEEK/](https://pyimagesearch.com/category/deepseek/))

MACHINE LEARNING ([HTTPS://PYIMAGESEARCH.COM/CATEGORY/MACHINE-LEARNING/](https://pyimagesearch.com/category/machine-learning/))

NEURAL NETWORKS ([HTTPS://PYIMAGESEARCH.COM/CATEGORY/NEURAL-NETWORKS/](https://pyimagesearch.com/category/neural-networks/))

TUTORIAL ([HTTPS://PYIMAGESEARCH.COM/CATEGORY/TUTORIAL/](https://pyimagesearch.com/category/tutorial/))

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

by **Puneet Mangla** (<https://pyimagesearch.com/author/puneetm/>) *on* March 23, 2026

[Home \(https://pyimagesearch.com/\)](https://pyimagesearch.com/) » [Blog \(https://pyimagesearch.com/blog/\)](https://pyimagesearch.com/blog/) »

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

Table of Contents

- **[DeepSeek-V3 from Scratch: Mixture of Experts \(MoE\)](#)**
- **[The Scaling Challenge in Neural Networks](#)**
- **[Mixture of Experts \(MoE\): Mathematical Foundation and Routing Mechanism](#)**
- **[SwiGLU Activation in DeepSeek-V3: Improving MoE Non-Linearity](#)**
- **[Shared Expert in DeepSeek-V3: Universal Processing in MoE Layers](#)**
- **[Auxiliary-Loss-Free Load Balancing in DeepSeek-V3 MoE](#)**
- **[Sequence-Wise Load Balancing for Mixture of Experts Models](#)**

- [Expert Specialization in MoE: Emergent Behavior in DeepSeek-V3](#)
- [Implementation: Building the DeepSeek-V3 MoE Layer from Scratch](#)
- [MoE Design Decisions in DeepSeek-V3: SwiGLU, Shared Experts, and Routing](#)
- [MoE Computational and Memory Analysis in DeepSeek-V3](#)
- [MoE Expert Specialization in Practice: Real-World Behavior](#)
- [Training Dynamics of MoE: Load Balancing and Expert Utilization](#)
- [Mixture of Experts vs Related Techniques: Switch Transformers and Sparse Models](#)
- [Summary](#)
 - [Citation Information](#)

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

In the first two parts of this series, we established the foundations of DeepSeek-V3 by implementing its core configuration and positional encoding, followed by a deep dive into **Multi-Head Latent Attention (MLA)**. Together, these components set the stage for a model that is both efficient and capable of handling long-range dependencies. With those building blocks in place, we now explore another key innovation in DeepSeek-V3: the **Mixture of Experts (MoE)**.



DeepSeek-V3 from Scratch: Mixture of Experts (MoE)



- Scaling transformers with Mixture of Experts
- Expert routing and sparse computation
- SwiGLU activation in MoE layers
- Load balancing and expert utilization
- Implement DeepSeek-V3 MoE layer



(<https://pyimagesearch.com/wp-content/uploads/2026/03/deepseek-v3-from-scratch-moe-featured.png>)

MoE introduces a dynamic way of scaling model capacity without proportionally increasing computational cost. Instead of activating every parameter for every input, the model selectively routes tokens through specialized “expert” networks, allowing it to expand representational power while keeping inference efficient. In

this lesson, we'll unpack the theory behind MoE, explain how expert routing works, and then implement it step by step. This installment continues our broader goal of reconstructing DeepSeek-V3 from scratch — showing how each innovation, from RoPE to MLA to MoE, fits together into a cohesive architecture that balances scale, efficiency, and performance.

This lesson is the 3rd in a 6-part series on **Building DeepSeek-V3 from Scratch**:

- 1 [**DeepSeek-V3 Model: Theory, Config, and Rotary Positional Embeddings**](https://pyimg.co/1atre)
(<https://pyimg.co/1atre>) [Click here to download the source code to this post](#)
- 2 [**Build DeepSeek-V3: Multi-Head Latent Attention \(MLA\) Architecture**](https://pyimg.co/scgjl)
(<https://pyimg.co/scgjl>)
- 3 [**DeepSeek-V3 from Scratch: Mixture of Experts \(MoE\)**](https://pyimg.co/a1w0g)
(<https://pyimg.co/a1w0g>) (this tutorial)
- 4 Lesson 4
- 5 Lesson 5

To learn about DeepSeek-V3 and build it from scratch, *just keep reading*.

The Scaling Challenge in Neural Networks

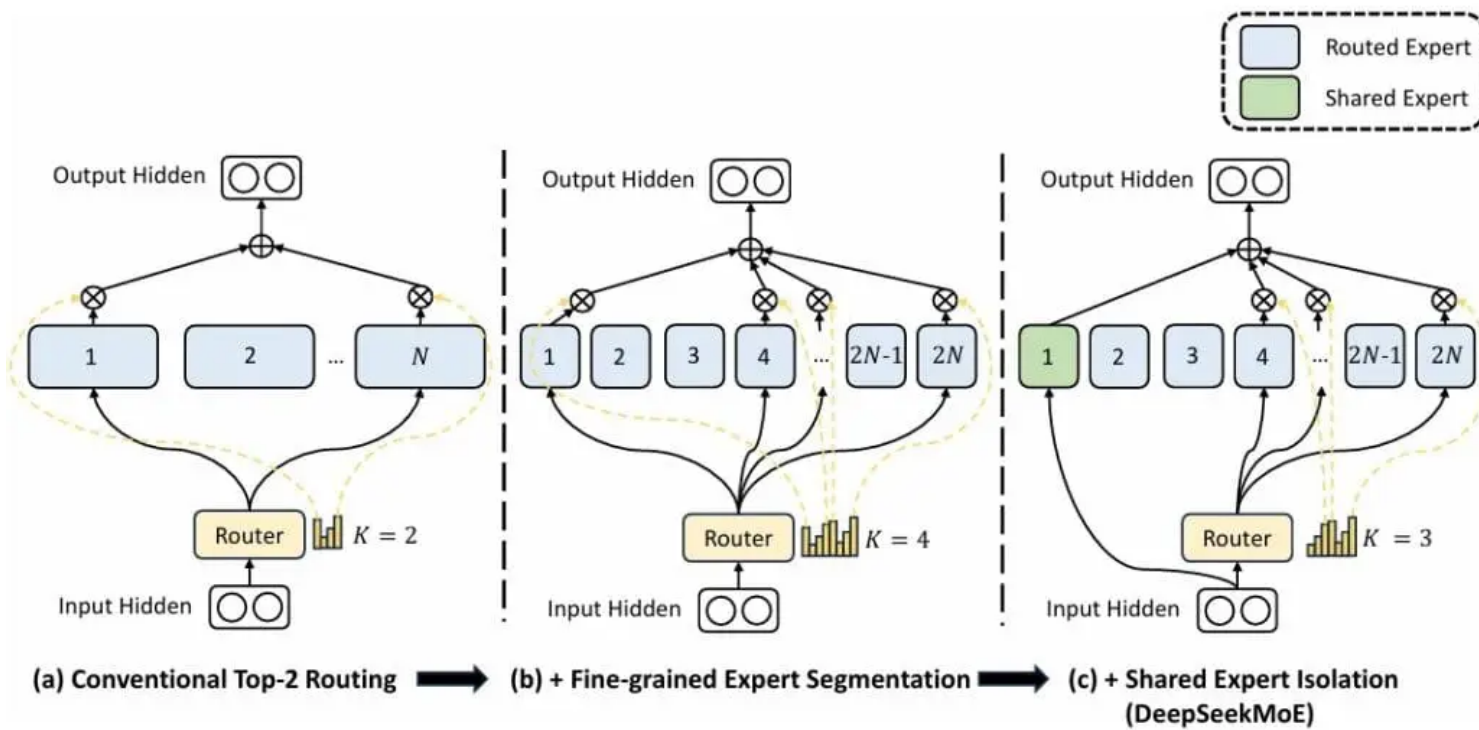
As we scale neural networks, we face a fundamental tradeoff: larger models have greater capacity to learn complex patterns, but they're more expensive to train and deploy. A standard Transformer feedforward layer applies the same computation to every token:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 ,$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ are weight matrices, typically with $d_{\text{ff}} = 4 \times d_{\text{model}}$. For our model with $d_{\text{model}} = 256$, this means $d_{\text{ff}} = 1024$, giving us approximately 256K parameters per FFN (FeedForward Network) per layer.

To increase model capacity, we could simply make d_{ff} larger — say, $8 \times d_{\text{model}}$ instead of $4\times$. This doubles the FFN parameters and theoretically doubles capacity. But it also doubles the computation for every token, even if most don't need that extra capacity.

Mixture of Experts (**Figure 1**) offers a more efficient scaling paradigm: instead of a single large FFN, we create multiple smaller expert FFNs and route each token to a subset of these experts. This gives us the capacity of a much larger model while maintaining computational efficiency.



(<https://pyimagesearch.com/wp-content/uploads/2026/03/image-9-scaled.jpeg>)

Figure 1: Types of Mixture of Experts Models (source: [Dai et al., 2024](https://arxiv.org/pdf/2401.06066) (<https://arxiv.org/pdf/2401.06066>)).

Mixture of Experts (MoE): Mathematical Foundation and Routing Mechanism

Consider N expert networks, each with the same architecture as a standard FFN:

$$E_i(x) = \text{SwiGLU}(x)$$

for $i = 1, \dots, N$. Instead of using all experts for every token, we select the top-k experts. The selection is determined by a learned routing function:

$$r(x) = \text{softmax}(xW_r + b) \in \mathbb{R}^N$$

where $W_r \in \mathbb{R}^{d_{\text{model}} \times N}$ is the router weight matrix and $b \in \mathbb{R}^N$ is a learnable bias vector. This gives us a probability distribution over experts for each token.

Top-k Routing: We select the top-k experts based on router probabilities:

$$\mathcal{T}_k(x) = \{i \mid r_i(x) \text{ is in the top-}k \text{ values of } r(x)\}$$

The final output combines the selected experts, weighted by their normalized routing probabilities:

$$\text{MoE}(x) = \sum_{i \in \mathcal{T}_k(x)} \frac{r_i(x)}{\sum_{j \in \mathcal{T}_k(x)} r_j(x)} E_i(x)$$

The renormalization $\frac{r_i(x)}{\sum_{j \in \mathcal{T}_k(x)} r_j(x)}$ ensures the selected experts' weights sum to 1.

Capacity and Computation: With $N = 4$ experts and $k = 2$ (our configuration), each token activates 2 out of 4 experts. If each expert has the same size as a standard FFN, we have 4× the parameters but only 2× the computation per token. This is the MoE efficiency advantage: parameter count scales with N , but computation scales with k .

SwiGLU Activation in DeepSeek-V3:

Improving MoE Non-Linearity

DeepSeek uses SwiGLU (Swish-Gated Linear Unit) instead of the traditional GELU (Gaussian Error Linear Units) activation. SwiGLU is a gated activation function that has shown superior performance in language models:

$$\text{SwiGLU}(x) = \text{SiLU}(\text{gate}(x)) \odot \text{up}(x)$$

where:

- $\text{gate}(x) = xW_{\text{gate}}$: projects input to hidden dimension
- $\text{up}(x) = xW_{\text{up}}$: is another projection to hidden dimension
- $\text{SiLU}(x) = x \cdot \sigma(x)$: is the Swish activation (smooth version of ReLU)
- $\odot$: denotes element-wise multiplication
- The result is then projected back: $\text{down}(\text{SwiGLU}(x))$

The gating mechanism allows the network to control information flow more precisely than simple activation functions. The SiLU activation provides smooth gradients everywhere, improving training dynamics compared to ReLU's hard

threshold.

Shared Expert in DeepSeek-V3: Universal Processing in MoE Layers

DeepSeek introduces a **shared expert** that processes all tokens in addition to the routed experts. This design addresses a key limitation of pure MoE: some computations are beneficial for all tokens regardless of their content.

$$\text{MoE}_{\text{total}}(x) = \text{SharedExpert}(x) + \sum_{i \in \mathcal{T}_k(x)} w_i E_i(x)$$

The shared expert has a larger hidden dimension (768 in our configuration vs 512 for individual experts) and processes every token. This ensures that:

- Common patterns are efficiently handled by dedicated capacity

- Specialized experts can focus on token-specific features
- Training is more stable with guaranteed gradient flow

The shared expert serves as a “base” computation that’s always present, while routed experts add specialized processing on top of it.

Auxiliary-Loss-Free Load Balancing in DeepSeek-V3 MoE

A critical challenge in MoE is load balancing. If the router learns to always send tokens to the same one or two experts, we lose the benefits of having multiple experts — the unused experts contribute nothing, and the overused ones become bottlenecks.

Traditional MoE models use an **auxiliary loss** that penalizes uneven expert usage:

$$\mathcal{L}_{\text{aux}} = \alpha \sum_{i=1}^N \left(\frac{L_i}{|\mathcal{B}|} - \frac{k}{N} \right)^2$$

where L_i is the number of tokens routed to expert i , $|\mathcal{B}|$ is batch size, and α is a coefficient. However, auxiliary losses add complexity and require careful tuning.

DeepSeek’s Innovation: Auxiliary-loss-free load balancing through **dynamic bias updates**. Instead of penalizing imbalance during training, we adjust the router biases to encourage balanced usage:

During training, we monitor how many tokens are routed to each expert. This gives us an `expert_usage` vector, where each entry counts the number of tokens assigned to a particular expert. We then compute the average usage across all experts.

To maintain a balanced load, we adjust the router biases: if an expert is used more than the average, its bias is decreased to make it less likely to be chosen in the future; if it is used less than the average, its bias is increased to make it more likely

to be selected. This dynamic bias update encourages fair distribution of tokens across experts without requiring an explicit auxiliary loss.

Let u_i denote the usage (number of tokens) of expert i , and let

$$\bar{u} = \frac{1}{N} \sum_{j=1}^N u_j$$

be the average usage across all N experts. The router bias for expert i , denoted b_i , is updated as:

$$b_i \leftarrow \begin{cases} b_i - \eta, & \text{if } u_i > \bar{u} \\ b_i + \eta, & \text{if } u_i \leq \bar{u} \end{cases},$$

where η is the learning rate controlling the magnitude of the bias adjustment.

This approach:

- Eliminates the need for auxiliary loss hyperparameter tuning
- Provides smoother load balancing over time

- Doesn't interfere with the primary task loss
- Automatically adapts to data distribution changes

The bias updates are performed with a small learning rate (0.001 in our implementation) to ensure gradual adjustment without disrupting training.

Sequence-Wise Load Balancing for Mixture of Experts Models

For even better load balancing, DeepSeek can use a **complementary sequence-wise auxiliary loss**. This encourages different sequences in a batch to use different experts:

$$\mathcal{L}_{\text{comp}} = \frac{1}{B^2} \sum_{i=1}^B \sum_{j \neq i}^B \text{sim}(u_i, u_j),$$

where u_i is the expert usage vector for sequence i (i.e., which experts were used), and sim measures similarity. By minimizing this loss, we encourage sequences to be complementary — if sequence A uses experts 1 and 2 heavily, sequence B should use experts 3 and 4.

Expert Specialization in MoE: Emergent Behavior in DeepSeek-V3

A fascinating property of MoE is expert specialization. Even though we don't explicitly tell experts what to specialize in, they often learn to handle different types of patterns. In language models, researchers have observed:

- **Syntactic experts:** Handle grammatical structures, verb conjugations
- **Semantic experts:** Process meaning, synonyms, and conceptual relationships

- **Domain experts:** Specialize in specific topics (e.g., scientific text, dialogue)
- **Numerical experts:** Handle arithmetic, dates, quantities

This specialization emerges naturally as the routing function learns which experts are most effective for different inputs. Gradient flow during training reinforces this — when an expert performs well on certain patterns, the router learns to send similar patterns to that expert.

Mathematically, we can think of each expert as learning a local model $E_i(x)$ that's particularly good in some region of the input space. The router function $r(x)$ implicitly partitions the input space, assigning different regions to different experts. This is similar to a mixture of experts in classical machine learning, but learned end-to-end through backpropagation.

Implementation: Building the DeepSeek-V3 MoE Layer from Scratch

Let's implement the complete MoE layer with expert networks, routing, and load balancing:

```
DeepSeek-V3 from Scratch: Mixture of Experts (MoE)
```

```
1. class SwiGLU(nn.Module):
2.     """SwiGLU activation function used in DeepSeek
   experts"""
3.
4.     def __init__(self, input_dim: int, hidden_dim: int,
   output_dim: int, bias: bool = True):
5.         super().__init__()
6.         self.gate_proj = nn.Linear(input_dim, hidden_dim,
   bias=bias)
```

```
7.         self.up_proj = nn.Linear(input_dim, hidden_dim,
bias=bias)
8.         self.down_proj = nn.Linear(hidden_dim, output_dim,
bias=bias)
9.
10.    def forward(self, x: torch.Tensor):
11.        gate = F.silu(self.gate_proj(x)) # SiLU
activation
12.        up = self.up_proj(x)
13.        return self.down_proj(gate * up)
```

Lines 1-13: SwiGLU Activation: The `SwiGLU` class implements a gated activation mechanism. We have 3 linear projections:

- `gate_proj` : for the gating signal
- `up_proj` : for the value branch

- `down_proj` : for the output projection

The forward pass applies SiLU (Sigmoid Linear Unit) to the gate projection, multiplies it element-wise with the up-projection, and projects back down. This creates a more expressive activation than simple GELU, with the gating mechanism allowing fine-grained control over information flow.

```
DeepSeek-V3 from Scratch: Mixture of Experts (MoE)
```

```
14. class MoEExpert(nn.Module):
15.     """Expert network for Mixture of Experts using
    SwiGLU"""
16.
17.     def __init__(self, config: DeepSeekConfig):
18.         super().__init__()
19.         self.expert_mlp = SwiGLU(
20.             config.n_embd,
21.             config.expert_intermediate_size,
```

```

22.         config.n_embd,
23.         config.bias
24.     )
25.
26.     def forward(self, x: torch.Tensor):
27.         return self.expert_mlp(x)

```

Lines 14-27: Expert with SwiGLU: Each `MoEExpert` is now a SwiGLU network instead of a simple FFN. The intermediate size (`expert_intermediate_size`) controls capacity — we use 512 in our configuration, which is smaller than the shared expert’s 768. This asymmetry reflects the fact that routed experts handle specialized patterns, while the shared expert handles common operations.

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

```

28. class MixtureOfExperts(nn.Module):
29.     """

```

```
30.         DeepSeek MoE layer with shared expert and auxiliary-
loss-free load balancing
31.
32.     Key features:
33.     - Shared expert that processes all tokens
34.     - Auxiliary-loss-free load balancing via bias updates
35.     - Top-k routing to selected experts
36.     """
37.
38.     def __init__(self, config: DeepSeekConfig):
39.         super().__init__()
40.         self.config = config
41.         self.n_experts = config.n_experts
42.         self.top_k = config.n_experts_per_token
43.         self.n_embd = config.n_embd
44.
45.
```

```
        # Router: learns which experts to use for each
token
46.         self.router = nn.Linear(config.n_embd,
config.n_experts, bias=False)
47.
48.         # Expert networks
49.         self.experts = nn.ModuleList([
50.             MoEExpert(config) for _ in
range(config.n_experts)
51.         ])
52.
53.         # Shared expert (processes all tokens)
54.         if config.use_shared_expert:
55.             self.shared_expert = SwiGLU(
56.                 config.n_embd,
57.                 config.shared_expert_intermediate_size,
```

```
58.         config.n_embd,  
59.         config.bias  
60.     )  
61.     else:  
62.         self.shared_expert = None  
63.  
64.         # Auxiliary-loss-free load balancing  
65.         self.register_buffer('expert_bias',  
torch.zeros(config.n_experts))  
66.         self.bias_update_rate = 0.001  
67.  
68.         self.dropout = nn.Dropout(config.dropout)
```

Lines 28-68: MoE Layer Structure: The `MixtureOfExperts` class orchestrates routing and expert execution. The 3 key additions:

- `shared_expert` : full-capacity expert that processes all tokens

- `expert_bias` : buffer for auxiliary-loss-free balancing
- `bias_update_rate` : controls how quickly biases adapt

The dropout provides regularization across the entire MoE output.

```
DeepSeek-V3 from Scratch: Mixture of Experts (MoE)
```

```
70.     def forward(self, x: torch.Tensor):
71.         batch_size, seq_len, hidden_dim = x.shape
72.         x_flat = x.view(-1, hidden_dim)
73.
74.         # Routing phase with bias for load balancing
75.         router_logits = self.router(x_flat) +
self.expert_bias
76.
77.         # Top-k routing
78.         top_k_logits, top_k_indices =
```

```
torch.topk(router_logits, self.top_k, dim=-1)
79.         routing_weights = torch.zeros_like(router_logits)
80.         routing_weights.scatter_(-1, top_k_indices,
F.softmax(top_k_logits, dim=-1))
81.
82.         # Expert computation
83.         output = torch.zeros_like(x_flat)
84.         expert_usage = torch.zeros(self.n_experts,
device=x.device)
```

Lines 70-84: Routing with Learnable Bias. The forward pass begins by flattening the input for efficient processing. We compute router logits and **add the expert bias** — this is the key to auxiliary-loss-free balancing. Overused experts have negative bias (making them less likely to be selected), while underused experts have positive bias (encouraging them to be selected). We then perform top-k selection and softmax normalization across the selected experts.

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

```
86.         # Process through selected experts
87.         for expert_idx in range(self.n_experts):
88.             expert_mask = (top_k_indices ==
expert_idx).any(dim=-1)
89.             expert_usage[expert_idx] =
expert_mask.sum().float()
90.
91.             if expert_mask.any():
92.                 expert_input = x_flat[expert_mask]
93.                 expert_output = self.experts[expert_idx]
(expert_input)
94.
95.                 # Weight by routing probability
96.                 weights = routing_weights[expert_mask,
expert_idx].unsqueeze(-1)
```

```
97.         output[expert_mask] += expert_output *
weights
98.
99.         # Add shared expert output (processes all tokens)
100.        if self.shared_expert is not None:
101.            shared_output = self.shared_expert(x_flat)
102.            output += shared_output
103.
104.        # Auxiliary-loss-free load balancing (update
biases during training)
105.        if self.training:
106.            with torch.no_grad():
107.                avg_usage = expert_usage.mean()
108.                for i in range(self.n_experts):
109.                    if expert_usage[i] > avg_usage:
110.                        self.expert_bias[i] -=
```

```
self.bias_update_rate
    else:
111.
112.         self.expert_bias[i] +=
self.bias_update_rate
113.
114.         output = self.dropout(output)
115.         return output.view(batch_size, seq_len,
hidden_dim), router_logits.view(batch_size, seq_len, -1)
```

Lines 86-97: Expert Processing. We iterate over all experts, identifying which tokens route to each one via the `expert_mask`. For each expert with assigned tokens, we extract those tokens, process them through the expert network, weight them by routing probability, and accumulate them into the output. This selective execution is what makes MoE efficient — we don't compute all experts for all tokens.

Lines 100-102: Shared Expert. The shared expert processes **all** tokens unconditionally and adds its output to the routed experts' output. This ensures every token receives some baseline processing, improving training stability and providing capacity for universal patterns. The shared expert's larger hidden dimension (768 vs 512) reflects its broader responsibility.

Lines 105-112: Auxiliary-Loss-Free Balancing. During training, we update expert biases based on usage. We compute average usage across experts, then adjust biases: overused experts receive negative adjustments (discouraging future selection), while underused experts receive positive adjustments (encouraging future selection). Using the `torch.no_grad()` context ensures these bias updates don't interfere with gradient computation. The small update rate (0.001) provides smooth, stable balancing over time.

Lines 114-115: Output and Return. We apply dropout to the combined output (routed + shared experts) and reshape back to the original dimensions. We return both the output and router logits — the latter can be used for optional auxiliary loss computation.

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

```
117.     def _complementary_sequence_aux_loss(self,  
router_logits, seq_mask=None):  
118.         """  
119.         router_logits: [batch_size, seq_len, num_experts]  
120.         Raw logits from the router before softmax.  
121.         seq_mask: optional mask for padding tokens.  
122.         """  
123.  
124.         # Convert to probabilities  
125.         probs = F.softmax(router_logits, dim=-1) # [B, T,  
E]  
126.  
127.         # Aggregate per-sequence expert usage  
128.         if seq_mask is not None:  
129.             probs = probs * seq_mask.unsqueeze(-1) # mask
```

```
padding
130.         seq_usage = probs.sum(dim=1) # [B, E]
131.
132.         # Normalize per sequence
133.         seq_usage = seq_usage / seq_usage.sum(dim=-1,
keepdim=True)
134.
135.         # Compute pairwise similarity between sequences
136.         sim_matrix = torch.matmul(seq_usage,
seq_usage.transpose(0, 1)) # [B, B]
137.
138.         # Encourage complementarity: minimize similarity
off-diagonal
139.         batch_size = seq_usage.size(0)
140.         off_diag = sim_matrix - torch.eye(batch_size,
device=sim_matrix.device)
```

```
141.         loss = off_diag.mean()  
142.  
143.         return loss
```

Lines 117-143: Complementary Sequence-Wise Loss. This method implements an alternative load-balancing approach. It converts router logits to probabilities, aggregates expert usage for each sequence, and computes pairwise similarity between sequences' expert usage patterns. By minimizing off-diagonal similarity, we encourage different sequences to use different experts, promoting diversity in expert utilization. This can be added to the training loss with a small weight (e.g., 0.01).

MoE Design Decisions in DeepSeek-V3:

SwiGLU, Shared Experts, and Routing

Several implementation choices merit discussion:

SwiGLU vs GELU: We use SwiGLU instead of traditional GELU because empirical research shows it consistently outperforms GELU in language models. The gating mechanism provides more expressive power, and SiLU's smoothness improves gradient flow. The computational cost is slightly higher (three projections instead of two), but the quality improvement justifies it.

Shared Expert Design: The shared expert is a DeepSeek innovation that addresses a key limitation of pure MoE: some computations benefit all tokens. By providing dedicated capacity for universal processing, we free routed experts to specialize more aggressively. The larger hidden dimension (768 vs 512) for the shared expert reflects empirical findings that shared capacity requires more parameters than individual experts.

Auxiliary-Loss-Free Balancing: Traditional MoE uses auxiliary losses, such as:

$$\mathcal{L}_{\text{aux}} = \alpha \cdot N \sum_{i=1}^N f_i \cdot P_i$$

where f_i is the fraction of tokens routed to expert i and P_i is the average routing probability. This requires tuning α (typically 0.01-0.1). Our bias-based approach eliminates the need for this hyperparameter, simplifying training. The tradeoff is that bias updates are less direct than gradient-based learning, but in practice, the smoother adaptation works well.

Complementary Sequence-Wise Loss: This alternative balancing approach is useful when batch diversity is high. By encouraging different sequences to use different experts, we naturally achieve balance. However, if the batch contains very similar sequences (e.g., all from the same domain), this loss may not be effective. It's best used in combination with bias-based balancing or as an optional auxiliary objective.

Expert Capacity: Production MoE systems often implement **expert capacity constraints** — if too many tokens route to one expert, excess tokens are dropped or routed to a second choice. We don't implement this in our educational model, but the formula would be:

$$\text{capacity}_i = \frac{|\mathcal{B}| \cdot k}{N} \cdot \text{factor}$$

where factor is typically 1.25-1.5. Tokens beyond this capacity are handled via overflow strategies.

MoE Computational and Memory Analysis in DeepSeek-V3

Let's analyze the computational cost. For a standard FFN with hidden dimension d_{ff} :

$$\text{FLOPs}_{\text{standard}} = 2 \cdot d_{\text{model}} \cdot d_{ff} + 2 \cdot d_{ff} \cdot d_{\text{model}} = 4 \cdot d_{\text{model}} \cdot d_{ff}$$

For our MoE with $N = 4$ routed experts (each with $d_{\text{expert}} = 512$), $k = 2$ selected, and shared expert ($d_{\text{shared}} = 768$):

$$\text{FLOPs}_{\text{MoE}} = d_{\text{model}} \cdot N + k \cdot \text{SwiGLU}_{\text{expert}} + \text{SwiGLU}_{\text{shared}}$$

The SwiGLU computation involves three projections:

$$\text{SwiGLU}_{\text{expert}} = 3 \cdot d_{\text{model}} \cdot d_{\text{expert}} + 3 \cdot d_{\text{expert}} \cdot d_{\text{model}} = 6 \cdot d_{\text{model}} \cdot d_{\text{expert}}$$

For our configuration:

- **Routing:** $256 \cdot 4$ (negligible)
- **Routed experts:** $2 \cdot 6 \cdot 256 \cdot 512 = 1,572,864$
- **Shared expert:** $6 \cdot 256 \cdot 768 = 1,179,648$
- **Total:** $\sim 2.75\text{M}$ FLOPs per token

Compare to a standard FFN with $d_{ff} = 1024$: $4 \cdot 256 \cdot 1024 = 1,048,576$ FLOPs. Our MoE uses $\sim 2.6\times$ more computation but has much higher capacity ($4 \text{ experts} \times 512 + 1 \text{ shared} \times 768 = 2,816$ vs $1,024$). We get $\sim 2.7\times$ capacity for $\sim 2.6\times$ computation — roughly linear scaling, which is the goal.

Memory usage during the forward pass stores activations for active experts only. During backpropagation, we need gradients for all experts (since routing is differentiable), yet the memory remains manageable. The bias vector is tiny (4 floats for 4 experts).

MoE Expert Specialization in Practice:

Real-World Behavior

While we can't demonstrate this in our small toy model, in larger-scale MoE models, expert specialization is observable through analysis of routing patterns.

Researchers have visualized which experts activate for different types of inputs, revealing clear specialization. For example:

- **Multilingual models:** Different experts handle different languages
- **Code models:** Some experts handle syntax, others semantics, others API patterns
- **Reasoning models:** Numerical experts for math, logical experts for inference, retrieval experts for factual recall

This specialization isn't programmed — it emerges from optimization. The routing function learns to partition the input space, and experts learn to excel in their assigned partitions. It's a beautiful example of how end-to-end learning can discover structured solutions.

Training Dynamics of MoE: Load Balancing and Expert Utilization

In practice, MoE training exhibits interesting dynamics:

Early Training: Routing is initially random or near-uniform. All experts receive a similar load. The shared expert learns basic patterns that benefit all tokens.

Mid Training: Routing starts specializing. Some experts become preferred for certain patterns. Load imbalance can emerge without careful management. Bias-based balancing begins correcting the imbalance.

Late Training: Experts are clearly specialized. Routing is confident (high softmax probabilities for selected experts). Load is balanced through continuous bias adjustment. The shared expert handles universal operations while routed experts focus on specialized patterns.

Monitoring expert usage during training is valuable. We can log:

- Per-expert selection frequency
- Routing entropy (higher means more uniform)
- Expert bias magnitudes (large values indicate strong correction needed)

Mixture of Experts vs Related Techniques:

Switch Transformers and Sparse Models

MoE shares ideas with several other architectural patterns:

Switch Transformers: Use top-1 routing (only one expert per token) for maximum efficiency. Simpler but less expressive than top-k.

Expert Choice: Instead of tokens choosing experts, experts choose tokens. Helps with load balancing but changes the computational pattern.

Sparse Attention: Like MoE, selectively activates parts of the network. Can be combined with MoE for extreme efficiency.

Dynamic Networks: Adapt network structure based on input. MoE is a specific form of dynamic computation.

With our MoE implementation complete, we've added efficient scaling to our model — the capacity grows superlinearly with computation cost. Combined with MLA's memory efficiency and the upcoming MTP's improved training signal, we're building a model that's efficient in training, efficient in inference, and capable of strong performance. Next, we'll tackle Multi-Token Prediction, which improves the training signal itself by having the model look further ahead.

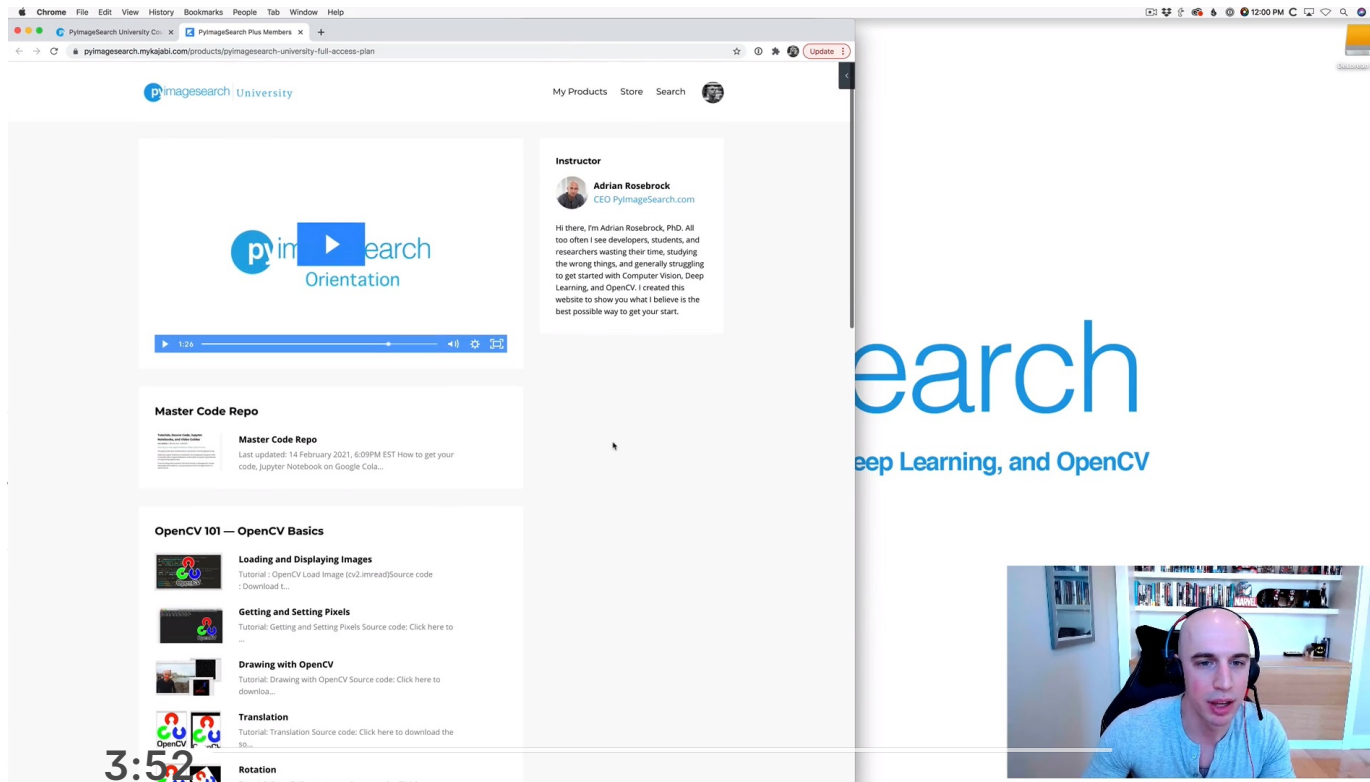
What's next? We recommend

PyImageSearch University

(https://pyimagesearch.com/pyimagesearch-university/?

utm_source=blogPost&utm_medium=bottom

Banner&utm_campaign=What%27s%20next%3F%20I%20recommend).



Do you think learning computer vision and deep learning has to be time-consuming, overwhelming, and complicated? Or has to involve complex mathematics and equations? Or requires a degree in computer science?

That's *not* the case.

All you need to master computer vision and deep learning is for someone to explain things to you in *simple, intuitive* terms. *And that's exactly what I do.* My mission is to change education and how complex Artificial Intelligence topics are taught.

If you're serious about learning computer vision, your next stop should be PyImageSearch University, the most comprehensive computer vision, deep learning, and OpenCV course online today. Here you'll learn how to *successfully* and *confidently* apply computer vision to your work, research, and projects. Join me in computer vision mastery.

Inside PyImageSearch University you'll find:

✓ **120+ courses** on essential computer vision, deep learning, and OpenCV topics

✓ **94+ Certificates** of Completion

✓ **115+ hours** of on-demand video

✓ **Brand new courses released *regularly***, ensuring you can keep up with state-of-the-art techniques

✓ **Pre-configured Jupyter Notebooks in Google Colab**

✓ Run all code examples in your web browser — works on Windows, macOS, and Linux (no dev environment configuration required!)

✓ Access to **centralized code repos for *all* 540+ tutorials** on PyImageSearch

✓ **Easy one-click downloads** for code, datasets, pre-trained models, etc.

✓ **Access** on mobile, laptop, desktop, etc.

**CLICK HERE TO JOIN PYIMAGESEARCH UNIVERSITY
([HTTPS://PYIMAGESEARCH.COM/PYIMAGESEARCH-
UNIVERSITY/?](https://pyimagesearch.com/pyimagesearch-university/?)**

**UTM_SOURCE=BLOGPOST&UTM_MEDIUM=BOTTOMBANNER&UT
M_CAMPAIGN=WHAT%27S%20NEXT%3F%20I%20RECOMMEND
)**

Summary

In the third installment of our **DeepSeek-V3 from Scratch** series, we turn our attention to the **Mixture of Experts (MoE)** framework, a powerful approach to scaling neural networks efficiently. We begin by unpacking the scaling challenge in modern architectures and how MoE addresses it through selective expert activation. From its mathematical foundation to the introduction of **SwiGLU activation**, we explore how enhanced non-linearity and universal shared experts contribute to more flexible and expressive models.

We then examine the mechanics of **load balancing**, highlighting innovations (e.g., auxiliary-loss-free balancing and complementary sequence-wise strategies). These techniques ensure that experts are used effectively without introducing

unnecessary complexity. We also explore how expert specialization emerges naturally during training, leading to diverse behaviors across experts that improve overall performance. This emergent specialization is not just theoretical — it becomes visible in practice, shaping how the model processes different types of input.

Finally, we walk through the **implementation of MoE**, discussing design decisions, computational trade-offs, and memory analysis. We connect these insights to related techniques, showing how MoE integrates into the broader landscape of efficient deep learning. By the end, we not only understand the theory but also gain practical knowledge of how to implement and optimize MoE within DeepSeek-V3. This part of the series equips us with the tools to harness expert specialization while keeping training dynamics balanced and efficient.

Citation Information

Mangla, P. “DeepSeek-V3 from Scratch: Mixture of Experts (MoE),” *PyImageSearch*, S. Huot, A. Sharma, and P. Thakur, eds., 2026, <https://pyimg.co/a1w0g> (<https://pyimg.co/a1w0g>)

DeepSeek-V3 from Scratch: Mixture of Experts (MoE)

```
@incollection{Mangla_2026_deepseek-v3-from-scratch-moe,  
  
  author = {Puneet Mangla},  
  
  title = {{DeepSeek-V3 from Scratch: Mixture of Experts  
(MoE)}},  
  
  booktitle = {PyImageSearch},  
  
  editor = {Susan Huot and Aditya Sharma and Piyush Thakur},  
  
  year = {2026},  
  
  url = {https://pyimg.co/a1w0g},
```

}

To download the source code to this post (and be notified when future tutorials are published here on PylmageSearch), *simply enter your email address in the form below!*



About the Author

I am currently working on Automated Form Conversion Service at Adobe Experience Manager (AEM) Forms Team. I love research (and did a bunch at IIT, where I studied). I have published in ECCV, WACV, ECML, CVPR-W, ICLR-W, and PRL Journal. I have also served as a reviewer for CVPR, ICLR,

WACV, ICML, TPAMI, and ECCV. Among other things, I am a binge-watcher, music, and a food lover.

Previous Article:

Build DeepSeek-V3: Multi-Head Latent Attention (MLA) Architecture

Next Article:

Autoregressive Model Limits and Multi-Token Prediction in DeepSeek-V3

(<https://pyimagesearch.com/2026/03/16/build-deepseek-v3-multi-head-latent-attention-mla-architecture/>) (<https://pyimagesearch.com/2026/03/30/autoregressive-model-limits-and-multi-token-prediction-in-deepseek-v3/>)

Comment section

Hey, Adrian Rosebrock here, author and creator of PyImageSearch. While I love hearing from readers, a couple years ago I made the tough decision to no longer offer 1:1 help over blog post comments.

At the time I was receiving 200+ emails per day and another 100+ blog post comments. I simply did not have the time to moderate and respond to them all, and the sheer volume of requests was taking a toll on me.

Instead, my goal is to *do the most good* for the computer vision, deep learning, and OpenCV community at large by focusing my time on authoring high-quality blog posts, tutorials, and books/courses.

If you need help learning computer vision and deep learning, I suggest you refer to my full catalog of books and courses (<https://pyimagesearch.com/books-and-courses/>) — they have helped tens

of thousands of developers, students, and researchers *just like yourself* learn Computer Vision, Deep Learning, and OpenCV.

Click here to browse my full catalog. (<https://pyimagesearch.com/books-and-courses/>)

Similar articles

ARUCO MARKERS OBJECT DETECTION
OPENCV TUTORIALS TUTORIALS

Determining ArUco marker type with OpenCV and Python

(<https://pyimagesearch.com/2020/12/28/determining-aruco-marker-type-with-opencv-and-python/>)

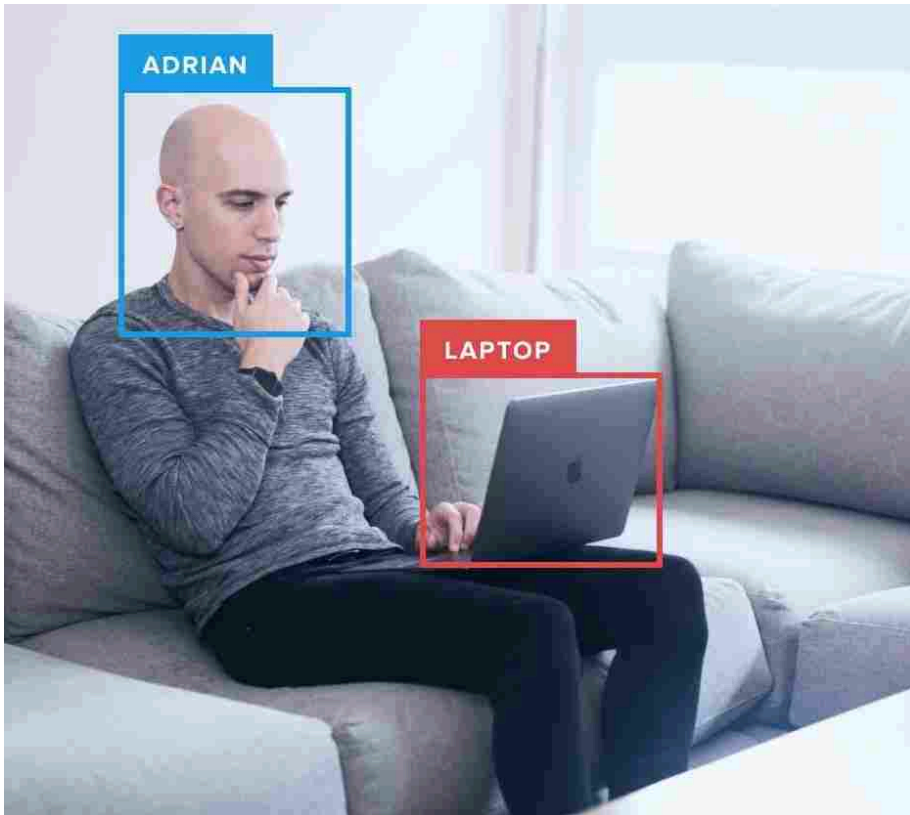
December 28, 2020

DEEP LEARNING PYTORCH
TUTORIALS

Intro to PyTorch: Training your first neural network using PyTorch

(<https://pyimagesearch.com/2021/07/12/intro-to-pytorch-training-your-first-neural-network-using-pytorch/>)

July 12, 2021



You can learn **Computer Vision**, **Deep Learning**, and **OpenCV**.

Get your FREE 17 page Computer Vision, OpenCV, and Deep Learning Resource Guide PDF. Inside you'll find our hand-picked tutorials, books, courses, and libraries to help you master CV and DL.

Topics

[Deep Learning](https://pyimagesearch.com/category/deep-learning/)

[\(https://pyimagesearch.com/category/deep-learning/\)](https://pyimagesearch.com/category/deep-learning/)

[Dlib Library](https://pyimagesearch.com/category/dlib/)

[\(https://pyimagesearch.com/category/dlib/\)](https://pyimagesearch.com/category/dlib/)

[Embedded/IoT and Computer Vision](https://pyimagesearch.com/category/embedded/)

[\(https://pyimagesearch.com/category/embedded/\)](https://pyimagesearch.com/category/embedded/)

[Face Applications](https://pyimagesearch.com/category/object-detection/)

[\(https://pyimagesearch.com/category/object-detection/\)](https://pyimagesearch.com/category/object-detection/)

[Machine Learning and Computer Vision](https://pyimagesearch.com/category/machine-learning/)

[\(https://pyimagesearch.com/category/machine-learning/\)](https://pyimagesearch.com/category/machine-learning/)

[Medical Computer Vision](https://pyimagesearch.com/category/medical/)
[\(https://pyimagesearch.com/category/medical/\)](https://pyimagesearch.com/category/medical/)

[Optical Character Recognition \(OCR\)](https://pyimagesearch.com/category/optical-character-recognition/)
[\(https://pyimagesearch.com/category/optical-character-recognition-ocr/\)](https://pyimagesearch.com/category/optical-character-recognition/)

[Object Detection](https://pyimagesearch.com/category/object-detection/)

Books & Courses

[PyImageSearch](https://pyimagesearch.com/university/)

[University](https://pyimagesearch.com/university/)
[\(https://pyimagesearch.com/pyimagesearch-university/\)](https://pyimagesearch.com/university/)

[FREE CV, DL, and OpenCV Crash Course](https://pyimagesearch.com/free-opencv-computer-vision-deep-learning-crash-course/)
[\(https://pyimagesearch.com/free-opencv-computer-vision-deep-learning-crash-course/\)](https://pyimagesearch.com/free-opencv-computer-vision-deep-learning-crash-course/)

[Practical Python and OpenCV](https://pyimagesearch.com/practical-python-and-opencv/)
[\(https://pyimagesearch.com/practical-python-and-opencv/\)](https://pyimagesearch.com/practical-python-and-opencv/)

PyImageSearch

[Affiliates](https://pyimagesearch.com/affiliates/)
[\(https://pyimagesearch.com/affiliates/\)](https://pyimagesearch.com/affiliates/)

[Get Started](https://pyimagesearch.com/start-here/)
[\(https://pyimagesearch.com/start-here/\)](https://pyimagesearch.com/start-here/)

[About](https://pyimagesearch.com/about/)
[\(https://pyimagesearch.com/about/\)](https://pyimagesearch.com/about/)

[Consulting](https://pyimagesearch.com/consulting-2/)
[\(https://pyimagesearch.com/consulting-2/\)](https://pyimagesearch.com/consulting-2/)

[Coaching](https://pyimagesearch.com/coaching/)

[m/category/faces/](https://pyimagesearch.com/category/faces/)

[Image Processing](https://pyimagesearch.com/category/image-processing/)

[\(https://pyimagesearch.com/category/image-processing/\)](https://pyimagesearch.com/category/image-processing/)

[Interviews](https://pyimagesearch.com/category/interviews/)

[\(https://pyimagesearch.com/category/interviews/\)](https://pyimagesearch.com/category/interviews/)

[Keras & Tensorflow](https://pyimagesearch.com/category/keras-and-tensorflow/)

[\(https://pyimagesearch.com/category/keras-and-tensorflow/\)](https://pyimagesearch.com/category/keras-and-tensorflow/)

[OpenCV Install Guides](https://pyimagesearch.com/opencv-tutorials-resources-guides/)

[\(https://pyimagesearch.com/opencv-tutorials-resources-guides/\)](https://pyimagesearch.com/opencv-tutorials-resources-guides/)

[\(https://pyimagesearch.com/category/object-detection/\)](https://pyimagesearch.com/category/object-detection/)

[Object Tracking](https://pyimagesearch.com/category/object-tracking/)

[\(https://pyimagesearch.com/category/object-tracking/\)](https://pyimagesearch.com/category/object-tracking/)

[OpenCV Tutorials](https://pyimagesearch.com/category/opencv/)

[\(https://pyimagesearch.com/category/opencv/\)](https://pyimagesearch.com/category/opencv/)

[Raspberry Pi](https://pyimagesearch.com/category/raspberry-pi/)

[\(https://pyimagesearch.com/category/raspberry-pi/\)](https://pyimagesearch.com/category/raspberry-pi/)

[m/practical-python-opencv/](https://pyimagesearch.com/practical-python-opencv/)

[Deep Learning for Computer Vision with Python](https://pyimagesearch.com/deep-learning-computer-vision-with-python/)

[\(https://pyimagesearch.com/deep-learning-computer-vision-python-book/\)](https://pyimagesearch.com/deep-learning-computer-vision-python-book/)

[PyImageSearch Gurus Course](https://pyimagesearch.com/pyimagesearch-gurus-course/)

[\(https://pyimagesearch.com/pyimagesearch-gurus/\)](https://pyimagesearch.com/pyimagesearch-gurus/)

[Raspberry Pi for Computer Vision](https://pyimagesearch.com/raspberry-pi-for-computer-vision/)

[\(https://pyimagesearch.com/raspberry-pi-for-computer-vision/\)](https://pyimagesearch.com/raspberry-pi-for-computer-vision/)

[\(https://pyimagesearch.com/consult-adrian/\)](https://pyimagesearch.com/consult-adrian/)

[FAQ](https://pyimagesearch.com/faqs/)

[\(https://pyimagesearch.com/faqs/\)](https://pyimagesearch.com/faqs/)

[YouTube](https://pyimagesearch.com/youtube/)

[\(https://pyimagesearch.com/youtube/\)](https://pyimagesearch.com/youtube/)

[Blog](https://pyimagesearch.com/topics/)

[\(https://pyimagesearch.com/topics/\)](https://pyimagesearch.com/topics/)

[Contact](https://pyimagesearch.com/contact/)

[\(https://pyimagesearch.com/contact/\)](https://pyimagesearch.com/contact/)

[Privacy Policy](https://pyimagesearch.com/privacy-policy/)

[\(https://pyimagesearch.com/privacy-policy/\)](https://pyimagesearch.com/privacy-policy/)